DOCUMENTO DE MATRIZ DE TRAZABILIDAD DE REQUERIMIENTOS Y CONTROL DE CAMBIOS (MTR)

Control de versiones

Nombre	Fecha	Razón del cambio	Autor	Ver.	Inicio	Fin	T. est.	T. real
STKD_MTR_V1_0_221026.pdf	22-Oct-2026	Definición de la cadena de trazabilidad desde el SRS hacia el diseño orientado a objetos, implementación de la Iteración 1 (MOD-01), avance inicial de la Iteración 2 (MOD-02) y registro formal de cambios técnicos.	AESA (Apoyo: KTCG, DAOJ, VMTS)	1.0	16:00	23:07	200 min	487 min

1. Introducción

En este documento se establece la Matriz de Trazabilidad de Requerimientos y la Bitácora de Control de Cambios para el Sistema de Ventas de Aparatos Electrónicos de Escritorio (**StockFlow**). Para incorporar los requerimientos del documento SRS (*STKD_SRS_V0_2_220926*) y la arquitectura modular (*STKD_DAS_V0_1_191026*) a la programación en Java Swing, cada requerimiento se ha convertido en comportamientos verificables, tareas de desarrollo, clases orientadas a objetos y casos de prueba.

El identificador único (**ID**) se conserva a lo largo de toda la cadena de ingeniería de software para responder de forma verificable qué componente atiende cada necesidad del cliente y cuál es el estado real de construcción al cierre de la **Iteración 1 (MOD-01)** e inicio de la **Iteración 2 (MOD-02)**.

2. Especificación de Decisiones Abiertas Antes de Programar

Previo a la codificación en Apache NetBeans, se precisaron los datos obligatorios, las reglas de negocio y los criterios verificables que detallan el comportamiento esperado de cada requerimiento funcional y no funcional:

ID	Datos y reglas que debían definirse	Criterio verificable en el sistema
RF-01.1	Atributos obligatorios del catálogo inicial de aparatos electrónicos y restricción de edición directa en las celdas de la tabla de mostrador.	Al iniciar FrameVista, la tabla carga los 10 productos oficiales mostrando código, nombre, categoría, precio en MXN y stock disponible, impidiendo modificar celdas manualmente.
RF-01.2	Regla de combinación entre el texto capturado en el buscador (por código o nombre) y la categoría elegida en el selector desplegable.	Al escribir en txtBusqueda o cambiar CBcategoria, la tabla filtra las coincidencias en tiempo real ignorando mayúsculas/minúsculas; al elegir "Todas" sin texto se muestran los 10 artículos.
RF-02.1	Comportamiento al agregar un artículo que ya se encuentra en el carrito de compra y validación de límite frente a la existencia física (RN-01, RN-02, EX-01).	Si el producto ya está en la lista de compra, se acumula la cantidad en la misma fila; si la cantidad total solicitada supera el stock del producto, se rechaza la operación y se muestra la alerta EX-01.
RF-02.2	Fórmula de desglose de impuestos (IVA del 16%) a partir del importe de las partidas y recálculo al quitar artículos o limpiar la lista.	Cada inserción, eliminación o limpieza del carrito actualiza en pantalla el Subtotal sin IVA, el IVA (16%) y el Total en MXN mediante CalculadoraImpuestos.
RF-03.1 a RF-03.3	Condición de lista no vacía (RN-04), estructura del resumen previo al cierre, descuento de inventario y formato del folio consecutivo (STKD-2026-XXXX).	El botón de confirmación permanece inactivo durante las pruebas de catálogo y preparación inicial, habilitándose para el flujo transaccional programado en la Iteración 3 (MOD-03).
RNF-08	Estandarización de principios de Programación Orientada a Objetos	La capa de vista (FrameVista) invoca únicamente a los servicios del controlador,

ID	Datos y reglas que debían definirse	Criterio verificable en el sistema
	(herencia, clase abstracta, interfaces, polimorfismo y encapsulamiento) y separación en capas.	mientras que el modelo aplica herencia desde Entidad y polimorfismo mediante Calculable y RepositorioProducto.

3. Conversión de cada Requerimiento en Trabajo de Desarrollo en Java

En la siguiente tabla se establece la correspondencia entre cada requerimiento del sistema, las tareas técnicas realizadas, los elementos construidos en los paquetes de Java y la prueba asociada para su validación:

ID	Tareas de programación	Elementos implementados en Java	Prueba asociada
RF-01.1	Diseñar la superclase abstracta Entidad, aplicar herencia en Producto, asociar Categoria y cargar el catálogo inicial en el repositorio.	Entidad, Producto, Categoria, RepositorioProducto, ProductoDAO.listar(), CatalogoService.listar()	CP-01: cargarCatalogoInicial ConDiezProductos()
RF-01.2	Implementar búsqueda por coincidencia parcial de código/nombre combinada con filtro por categoría y enlazar eventos en tiempo real en la interfaz.	CatalogoService.buscar(), ProductoDAO.buscarCodigo(), eventos en txtBusqueda y CBcategoria (FrameVista)	CP-02: filtrarProductosPorTex toYCategoria()
RF-02.1	Capturar cantidad positiva en el control JSpinner, verificar	Producto.hayStock(), VentaService.validarStock(),	CP-03: rechazarArticuloCuan

ID	Tareas de programación	Elementos implementados en Java	Prueba asociada
	disponibilidad física con hayStock() y acumular partidas en el carrito.	VentaService.agregarAlCarrito()	doSuperaStockDisponible()
RF-02.2	Implementar la interfaz Calculable en DetalleVenta, gestionar baja de partidas y centralizar el desglose fiscal del 16%.	Calculable, DetalleVenta.calcularImporte(), CalculadoraImpuestos, VentaService	CP-04: recalcularSubtotalIvaYTotalAlModificarCarrito()
RF-03.1 a RF-03.3	Construir entidad Venta, diálogo de confirmación, persistencia atómica y generador de folios consecutivos (alcance de la Iteración 3).	Venta, PersistenciaVenta, VentaDAO, FolioGeneratorDAO *(Programados para Iteración 3)*	CP-05 y CP-06: confirmarVentaYGenerarFolioUnico()
RNF-08	Estructurar paquetes independientes (vista, controlador, modelo, dao) y desacoplar el acceso a datos mediante interfaces.	Paquetes com.stockflow.vista, controlador, modelo y dao	PR-01: Revisión de arquitectura y desacoplamiento por capas

4. Cadena de Trazabilidad Completa: Caso RF-01.2 y RF-02.1

A continuación se detalla la cadena completa de trazabilidad para el flujo de búsqueda en el catálogo e inserción validada en el carrito de venta (Requerimiento → Historia / Necesidad → Regla de Negocio → Tareas → Código → Pruebas):

Etapa de la Cadena	Resultado en el Proyecto StockFlow
Requerimiento	**RF-01.2:** Permitir buscar productos por código o nombre y filtrarlos por categoría. **RF-02.1:** Permitir agregar artículos a la lista de compra validando la existencia disponible.
Necesidad del usuario	Como vendedor de mostrador, quiero localizar rápidamente un aparato electrónico por su código o categoría y agregarlo a la lista de compra comprobando su stock, para atender al cliente con agilidad sin comprometer mercancía agotada.
Reglas de negocio	**RN-01:** La cantidad solicitada debe ser un número entero mayor que cero. **RN-02 / EX-01:** No se puede agregar al carrito una cantidad superior al stock físico disponible del producto.
Tareas de desarrollo	Crear jerarquía Entidad → Producto; programar filtrado en CatalogoService; validar stock en VentaService; actualizar tabla del carrito y etiquetas de impuestos en FrameVista.
Código implementado	Métodos CatalogoService.buscar(filtroTexto, categoria), Producto.hayStock(cantidad) y VentaService.agregarAlCarrito(producto, cantidad).
Pruebas de verificación	1. Filtrar por categoría "Perifericos" y comprobar que solo aparezcan 3 productos. 2. Agregar 2 unidades de PER-001 (stock 12) y verificar importe de $1,700.00. 3. Intentar agregar 1 unidad de ACC-010 (stock 0) y comprobar que se muestre la alerta EX-01 sin alterar el carrito.

Flujo lógico principal en el código (VentaService.java)

```
public String agregarAlCarrito(Producto p, int cantidad) {
    if (p == null) return "Seleccione un producto.";
    if (cantidad <= 0) return "La cantidad debe ser mayor a cero (RN-01).";

    DetalleVenta existente = null;
    for (DetalleVenta d : carrito) {
        if (d.getProducto().getIdentificador().equals(p.getIdentificador())) {
            existente = d;
            break;
        }
    }

    int totalSolicitado = cantidad + (existente != null ? existente.getCantidad() : 0);
    if (!validarStock(p, totalSolicitado)) {
        return "EX-01: Stock insuficiente. Disponible: " + p.getStock();
    }

    if (existente != null) {
        existente.actualizarCantidad(totalSolicitado);
    } else {
        carrito.add(new DetalleVenta(p, cantidad));
    }
    return null;
}
```

5. Matriz de Trazabilidad General (Corte al 22 de octubre de 2026)

En la siguiente matriz se registra el estado actual del desarrollo considerando el cierre de la **Iteración 1 (MOD-01)** y el avance inicial construido de la **Iteración 2 (MOD-02)**:

ID	Módulo / Especificación	Tareas EDT	Clases y Métodos en Java	Pruebas	Estado
RF-01.1	MOD-01 · Consulta de inventario inicial	1.3.1.1, 1.3.1.2	Entidad, Producto, ProductoDAO.listar(), FrameVista	CP-01 (PU1)	**Completado (Iteración 1)**
RF-01.2	MOD-01 · Búsqueda y filtrado por categoría	1.3.1.1, 1.3.1.2	RepositorioProducto, CatalogoService.buscar()	CP-02 (PU1)	**Completado (Iteración 1)**
RF-02.1	MOD-02 · Selección de cantidad y validación de stock (EX-01)	1.3.2.1	Producto.hayStock(), VentaService.validarStock(), agregarAlCarrito()	CP-03 (PU2)	**En progreso (Avance Iter. 2)**
RF-02.2	MOD-02 · Gestión del carrito y cálculo de Subtotal/IVA	1.3.2.1	Calculable, DetalleVenta.calcularImporte(), CalculadoraImpuestos	CP-04 (PU2)	**En progreso (Avance Iter. 2)**
RF-03.1	MOD-03 · Resumen de confirmación de compra	1.3.3.1	Venta.mostrarResumen(), btn_Confirmar (inactivo en Iter. 1-2)	CP-05 (PU3)	**Pendiente (Iteración 3)**

ID	Módulo / Especificación	Tareas EDT	Clases y Métodos en Java	Pruebas	Estado
RF-03.2	MOD-03 · Registro de venta y descuento de stock	1.3.3.1	PersistenciaVenta, VentaDAO.registrarVenta()	CP-06 (PU3)	**Pendiente (Iteración 3)**
RF-03.3	MOD-03 · Generación de folio único consecutivo	1.3.3.1	FolioGeneratorDAO.generarFolio()	CP-07 (PU3)	**Pendiente (Iteración 3)**
RNF-01	Usabilidad de mostrador en una sola ventana	1.2.6, 1.3.1.1	FrameVista (paneles de catálogo y carrito integrados)	PR-01	**Completado**
RNF-08	Arquitectura modular en 3 capas y POO	1.2.7, 1.3.1.1	Paquetes vista, controlador, modelo, dao	PR-02	**Completado**

6. Bitácora de Control de Cambios Técnicos y de Diseño (STKD_CCM)

Durante el desarrollo de la Iteración 1 y la preparación de la base de la Iteración 2 se registraron y evaluaron los siguientes ajustes técnicos para alinear el código fuente con los diagramas de clases del documento de arquitectura (*STKD_DAS_V0_1_191026*), sin alterar el alcance funcional acordado en el SRS:

Folio	Fecha	Elemento Afectado	Descripción del Cambio	Justificación e Impacto en Trazabilidad	Estado / Autor
CAM-01	18/10/2026	Paquete com.stockflow.modelo (Entidad, Producto)	Se incorporó la clase abstracta Entidad con el atributo protegido identificador y el método abstracto mostrarResumen(), estableciendo herencia hacia la clase Producto.	Aplicar herencia y abstracción de forma limpia en el modelo de dominio cumpliendo con el Diagrama de Clases 1 y el requisito RNF-08.	**Aprobado** — DAOJ / AESA
CAM-02	19/10/2026	com.stockflow.modelo.Categoria y Producto	Se reemplazó el manejo de la categoría como texto simple por una relación de composición hacia un objeto de la clase Categoria.	Mejorar el encapsulamiento de los datos del catálogo (RF-01.1) y mantener consistencia con el filtro de categorías en CatalogoService.	**Aprobado** — AESA
CAM-03	20/10/2026	RepositorioProducto y ProductoDAO	Se introdujo la interfaz RepositorioProducto entre CatalogoService y ProductoDAO para desacoplar las consultas del catálogo respecto al almacenamiento.	Facilitar las pruebas de las Iteraciones 1 y 2 en memoria antes de integrar el motor de base de datos local en la Iteración 4 (MOD-04).	**Aprobado** — VMTS / AESA

Folio	Fecha	Elemento Afectado	Descripción del Cambio	Justificación e Impacto en Trazabilidad	Estado / Autor
CAM-04	21/10/2026	Calculable, DetalleVenta y CalculadoraImpuestos	Se creó la interfaz Calculable con el método calcularImporte() y se separó el desglose del IVA (16%) dentro de la clase CalculadoraImpuestos.	Aplicar polimorfismo en el cálculo de partidas y evitar fórmulas matemáticas dentro de la ventana FrameVista (RF-02.2, RNF-08).	**Aprobado** — AESA
CAM-05	22/10/2026	CatalogoService.buscar() y FrameVista	Se ajustó el algoritmo de filtrado para evaluar de manera simultánea el texto escrito en txtBusqueda y la opción activa en CBcategoria.	Garantizar que la búsqueda en tiempo real conserve el filtro de categoría seleccionado durante las pruebas unitarias STKD_PU1 (RF-01.2).	**Aprobado** — KTCG / AESA

7. Conclusión

Mediante esta Matriz de Trazabilidad y Bitácora de Control de Cambios queda demostrado que el 100% de los requerimientos correspondientes a la Iteración 1 (MOD-01: Inventario y catálogo) han sido implementados y verificados en código Java bajo los principios de Programación Orientada a Objetos definidos en la arquitectura, contando además con el avance estructural de la Iteración 2 (MOD-02: Preparación de la venta y cálculo de impuestos) listo para completar su fase de pruebas.